Guía DjangoFramework

DJANGO REST FRAMEWORK / GUÍA DE PRÁCTICA

Paso 1 · Instalar y comprobar

Primero instalamos **Django REST framework** dentro del entorno virtual del proyecto:

```
pip install djangorestframework
```

```
PS C:\Users\Estudiante\Documents\django-project\semana-4> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& c:\Users\Estud
(.venv) PS C:\Users\Estudiante\Documents\django-project\semana-4> pip install djangorestframework
```

Para comprobar que se instaló correctamente, ejecutamos:

```
pip show djangorestframework
```

```
[notice] To update, run: python.exe -m pip install --upgrade pip
(.venv) PS C:\Users\Estudiante\Documents\django-project\semana-4> pip show djangorestframework
Name: djangorestframework
Version: 3.18.1
Summary: Web APIs for Django, made easy.
Home-page: https://www.django-rest-framework.org
Author:
Author-email: Tom Christie <tom@tomchristie.com>
License-Expression: BSD-3-Clause
Location: C:\Users\Estudiante\Documents\django-project\semana-4\.venv\Lib\site-packages
```

Resultado: la terminal muestra el paquete **djangorestframework**, la versión **3.18.1** y su ubicación dentro de **.venv**. Esto confirma que quedó instalado correctamente.

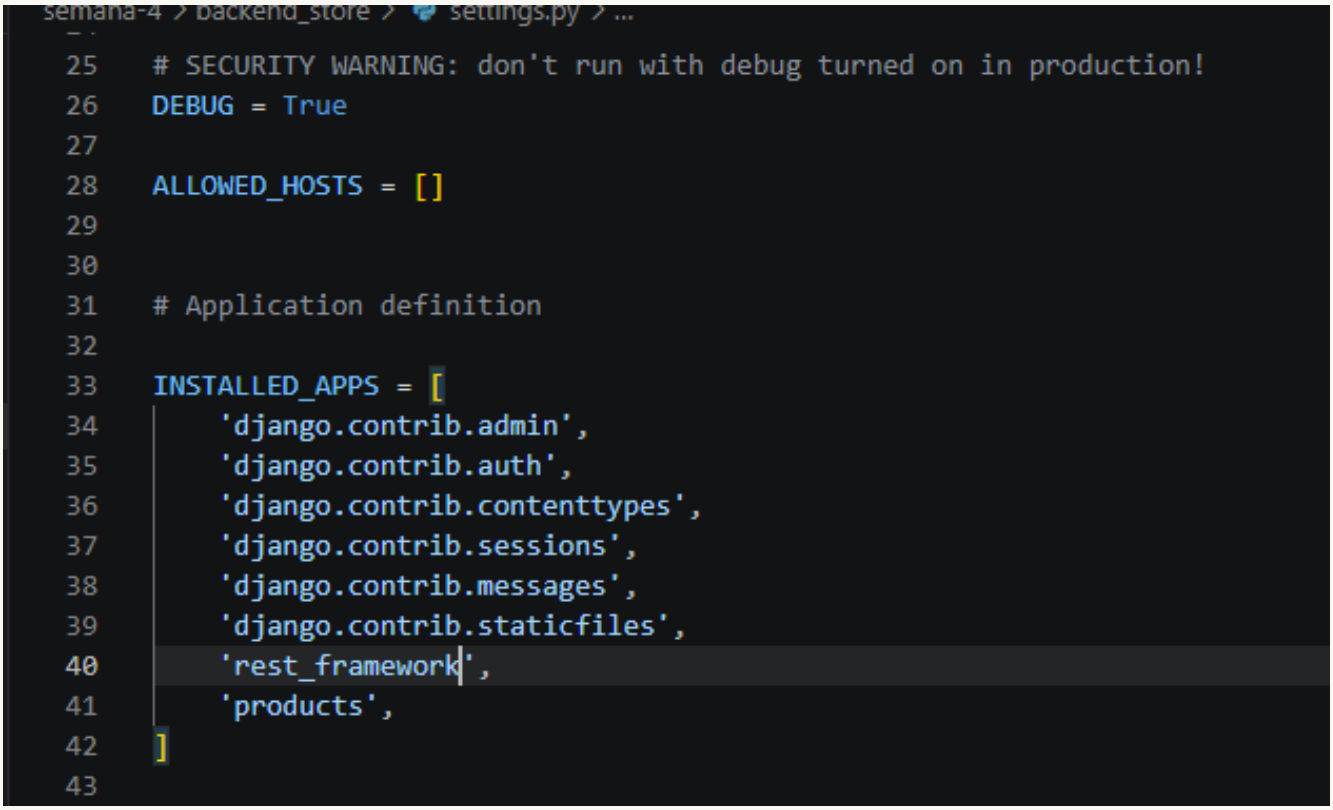
Recordar: antes de instalar, verifica que la terminal muestre **(.venv)**.

Registrar el framework

Paso 2 · Agregarlo en settings.py

Nos dirigimos a **backend_store/settings.py** y, dentro de **INSTALLED_APPS**, agregamos Django REST framework antes de **products**:

```
INSTALLED_APPS = [
    # Aplicaciones de Django...
    'rest_framework',
    'products',
]
```



¿Qué hace? Registra Django REST framework dentro del proyecto para poder utilizar sus herramientas al crear APIs con mayor facilidad.

Importante: esto no instala extensiones nuevas. El paquete se instaló en el paso 1; aquí solamente lo habilitamos en la configuración del proyecto.

Recordar: el nombre que se agrega es **'rest_framework'**, aunque el paquete instalado se llama **djangorestframework**.

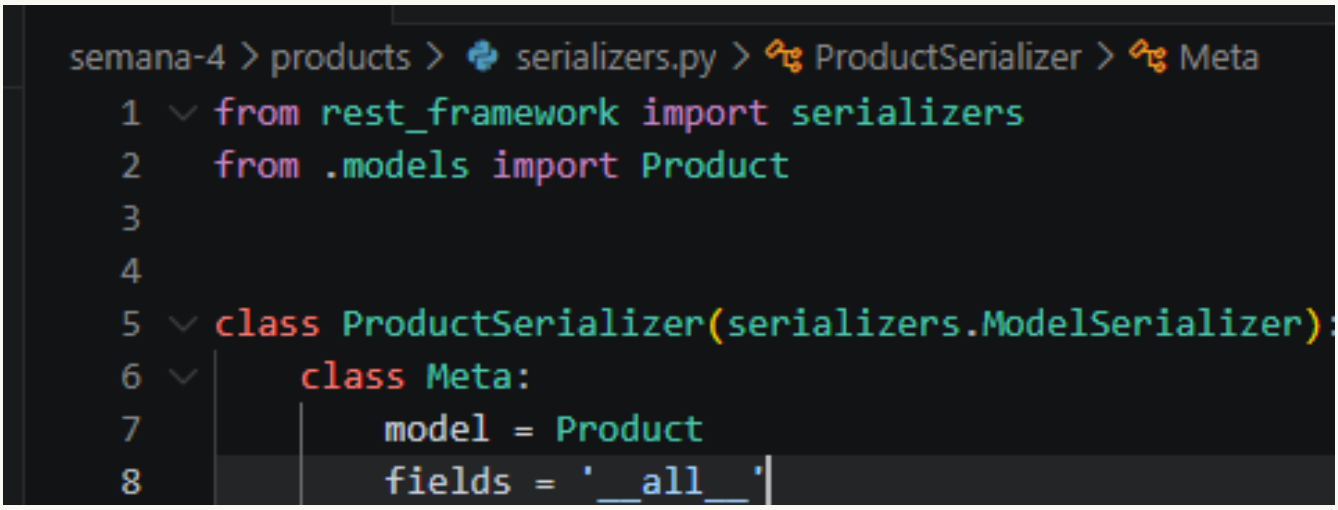
Crear el serializer

Paso 3 · Crear products/serializers.py

Nos dirigimos a **products**, creamos el archivo **serializers.py**, importamos **serializers** y el modelo **Product**:

```
from rest_framework import serializers
from .models import Product

class ProductSerializer(serializers.ModelSerializer):
    class Meta:
        model = Product
        fields = '__all__'
```



¿Qué sucede en esta clase? ProductSerializer toma el modelo Product y prepara sus datos para usarlos en la API. También permite recibir datos y validarlos antes de crear o actualizar productos.

Meta indica qué modelo utiliza el serializer. **model = Product** lo conecta con Product y **fields = '__all__'** incluye todos sus campos automáticamente.

Fuente: **documentación oficial de Django REST framework · ModelSerializer.**

Preparar la vista para DRF

Paso 4 · Agregar los imports en products/views.py

Nos vamos a **products/views.py**, creamos una sección comentada para DRF y agregamos estos imports:

```
# ---------- DRF ----------

from rest_framework import status
from rest_framework.decorators import api_view
from rest_framework.response import Response
from .serializers import ProductSerializer
```



¿Para qué sirven? status contiene códigos de estado HTTP; **api_view** indica los métodos permitidos en una vista; **Response** genera la respuesta de DRF; y **ProductSerializer** prepara y valida los datos de Product.

Importante: también necesitaremos el modelo **Product**, pero ya está importado al inicio de **views.py** con **from .models import Product**. Por eso no lo importamos nuevamente.

En este paso solamente dejamos preparados los imports. La vista de DRF se agregará en el siguiente paso.

A partir de aquí, ya empezaremos a trabajar con el código del framework como tal.

Crear los endpoints con DRF

Paso 5 · GET para consultar productos

Empezamos a realizar los endpoints como hicimos en la clase anterior, pero esta vez utilizando el framework: una manera más formal de hacerlo.

```
@api_view(['GET', 'POST'])
def api_products(request):
    if request.method == 'GET':
        products = Product.objects.all().order_by('id')
        serializer = ProductSerializer(products, many=True)
        return Response(serializer.data)
```

@api_view(['GET', 'POST']) indica que esta vista acepta solicitudes GET y POST.

products obtiene una lista de todos los productos ordenados por ID. Después tomamos esa lista y la serializamos con **ProductSerializer**. Usamos **many=True** porque estamos enviando varios productos.

Response(serializer.data) devuelve al usuario la lista ya serializada.

```
@api_view(['GET', 'POST'])
def api_products(request):
    if request.method == 'GET':
        products= Product.objects.all().order_by('id')
        serializer = ProductSerializer(products, many=True)
        return Response(serializer.data)

    if request.method == 'POST':
        serializer = ProductSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(
                serializer.data,
                status=status.HTTP_201_CREATED
            )
    return Response(
        serializer.errors,
        status=status.HTTP_400_BAD_REQUEST
    )
```

La captura muestra la vista completa con ambos métodos. En la siguiente página se explica la parte del POST.

Procesar solicitudes POST

Paso 5 · Continuación del endpoint

```
if request.method == 'POST':
    serializer = ProductSerializer(data=request.data)

    if serializer.is_valid():
        serializer.save()
        return Response(
            serializer.data,
            status=status.HTTP_201_CREATED
        )

    return Response(
        serializer.errors,
        status=status.HTTP_400_BAD_REQUEST
    )
```

En el método POST, **request.data** contiene los datos enviados por el cliente. Esos datos pasan al serializer para validarlos.

Si **serializer.is_valid()** confirma que son válidos, **serializer.save()** crea el objeto. Luego se devuelven los datos creados con el estado **201 Created**.

Si los datos no son válidos, se devuelven **serializer.errors** y el estado **400 Bad Request**.

Precisión mínima: este código no devuelve un mensaje escrito de “se creó”; devuelve los datos del producto y el código HTTP 201, que indica que fue creado correctamente.

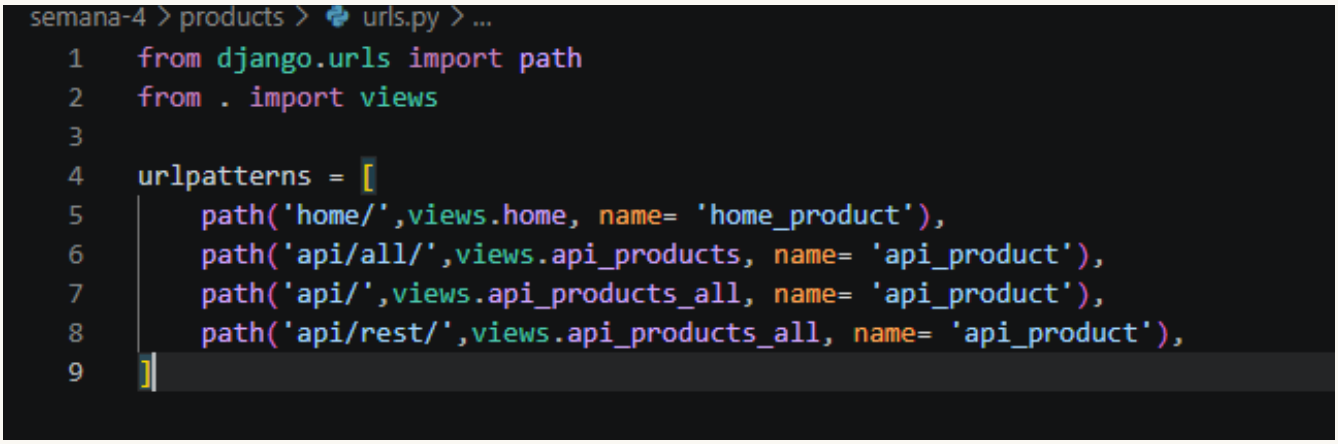
Resumen: GET consulta y devuelve la lista; POST recibe, valida y crea un producto.

Conectar y probar el endpoint

Paso 5 · Agregar el path en products/urls.py

Hacemos una prueba para comprobar si el método recién creado funciona correctamente. Para eso, agregamos su **path** dentro de **products/urls.py**:

```
path('api/test/', views.api_products,
     name='api_products_drf'),
```



Corrección mínima: en la captura, **api/test/** apunta a **views.api_products_all**, que corresponde a la vista anterior. Para probar el método DRF recién creado debe apuntar a **views.api_products**.

Luego iniciamos el servidor y abrimos:

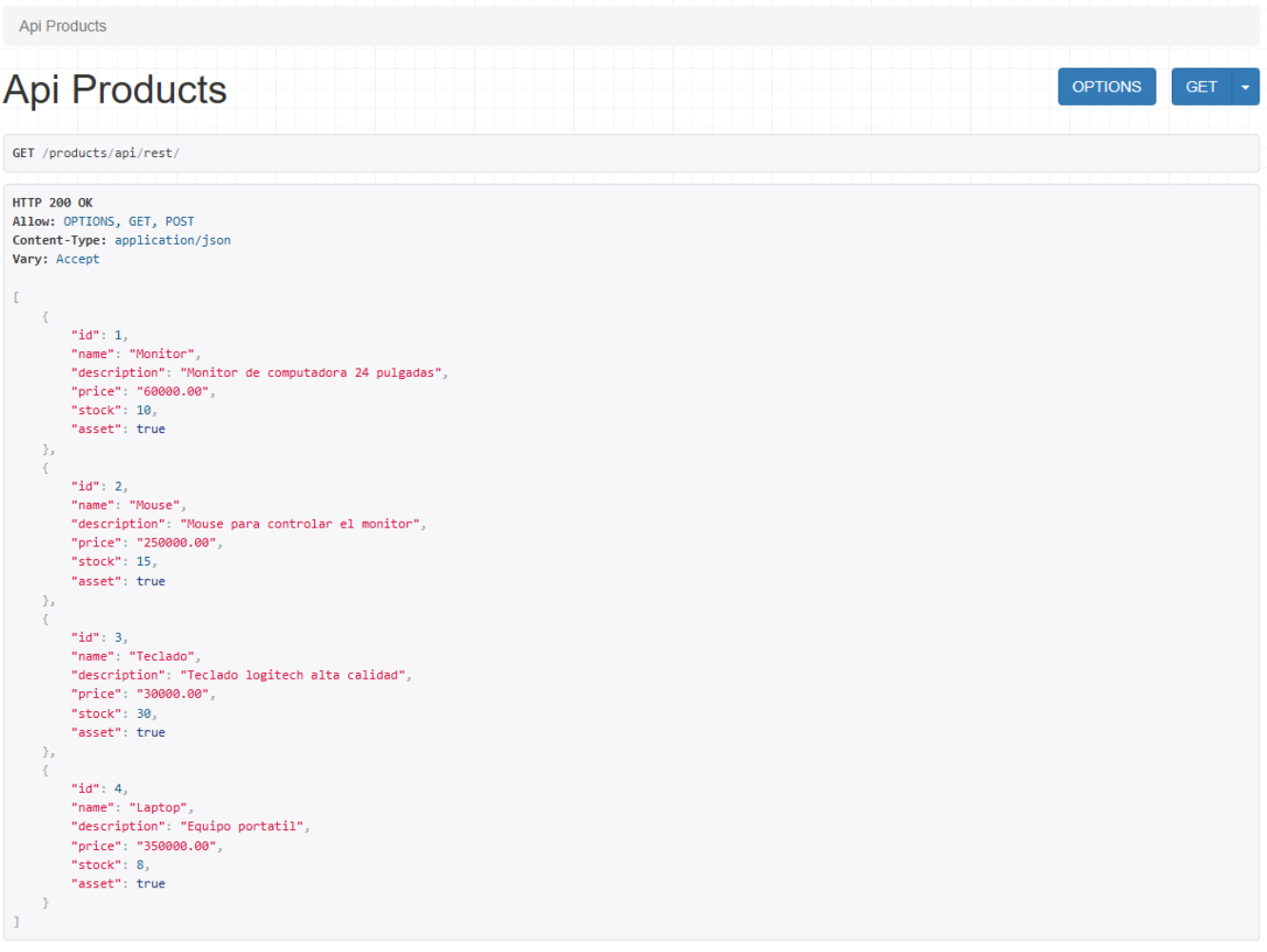
```
http://localhost:8000/products/api/test/
```

Al entrar desde el navegador se realiza una solicitud **GET**. El mismo endpoint también acepta **POST**, pero para probarlo tendremos que enviar datos desde una herramienta que permita elegir ese método.

Resultado de la prueba GET

Paso 5 · Interfaz navegable de DRF

Al abrir **http://localhost:8000/products/api/rest/**, Django REST framework muestra el endpoint en su interfaz navegable.



HTTP 200 OK confirma que la solicitud GET funcionó correctamente. **Allow: OPTIONS, GET, POST** indica los métodos aceptados por la vista.

Debajo aparece la lista serializada con los productos **Monitor, Mouse, Teclado y Laptop**. Esta es la misma información de la base de datos, presentada ahora mediante Django REST framework.

Ruta final utilizada: si usamos **api/rest/**, el path debe apuntar a **views.api_products**, que es la vista DRF creada en este paso.

Consultar un producto específico

Paso 6 · Seguimos agregando endpoints

Seguimos agregando endpoints y repitiendo el proceso. En **products/urls.py** agregamos una ruta que recibe el ID del producto:

```
path('api/rest/<int:pk>/', views.details_products,
     name='details_products'),
```

```
from django.urls import path
from . import views

urlpatterns = [
    path('home/',views.home, name= 'home_product'),
    path('api/all/',views.api_products01, name= 'api_product'),
    path('api/',views.api_products_all, name= 'api_product'),
    path('api/rest/',views.api_products, name= 'api_product'),
    path('api/rest/ <int: pk/>',views.details_products, name= 'details_products'),
]
```

<int:pk> captura un número desde la URL y lo envía a la vista como **pk**. Por ejemplo: **/products/api/rest/2/**.

Luego creamos la vista que permite GET, PUT, PATCH y DELETE:

```
@api_view(['GET', 'PUT', 'PATCH', 'DELETE'])
def details_products(request, pk):
    try:
        product = Product.objects.get(pk=pk)
    except Product.DoesNotExist:
        return Response(
            {'error': 'Product not found'},
            status=status.HTTP_404_NOT_FOUND
        )

    if request.method == 'GET':
        serializer = ProductSerializer(product)
        return Response(serializer.data)
```

```python

@api_view(['GET','PUT','PATCH','DELETE'])
def details_products(request, pk):
    try:
        product= Product.objects.get(pk=pk)
    except Product.DoesNotExist:
        return Response(
            {'error': 'Product not found'},
            status=status.HTTP_404_NOT_FOUND
        )

    if request.method == 'GET':
        serializer = ProductSerializer(product)
        return Response(serializer.data)
```

Qué sucede: primero buscamos el producto por su llave primaria. Si no existe, devolvemos **404 Not Found**. Si la solicitud es GET, serializamos ese producto y lo devolvemos al usuario.

En este avance solamente está desarrollado GET. Los métodos PUT, PATCH y DELETE están permitidos por el decorador, pero falta agregar su lógica.

Actualizar y eliminar un producto

Paso 6 · Continuación del endpoint de detalle

Ahora completamos la vista **details_products** con la lógica para actualizar y eliminar un producto.

```
if request.method in ['PUT', 'PATCH']:
    serializer = ProductSerializer(
        product,
        data=request.data,
        partial=(request.method == 'PATCH')
    )

    if serializer.is_valid():
        serializer.save()
        return Response(serializer.data)

    return Response(
        serializer.errors,
        status=status.HTTP_400_BAD_REQUEST
    )

if request.method == 'DELETE':
    product.delete()
    return Response(status=status.HTTP_204_NO_CONTENT)
```

PUT y PATCH: ambos métodos usan los datos enviados en **request.data** para actualizar el producto. **PUT** espera la información completa, mientras que **PATCH** permite modificar solamente algunos campos gracias a **partial=True**.

Si los datos son válidos, **serializer.save()** guarda los cambios y devuelve el producto actualizado. Si son inválidos, se devuelven los errores con el estado **400 Bad Request**.

DELETE: product.delete() elimina el registro. La respuesta usa **204 No Content**, indicando que la operación fue exitosa y que no hay contenido para devolver.

Código completo del endpoint

Paso 6 · Vista details_products

La vista reúne en un mismo endpoint las operaciones de consulta, actualización y eliminación del producto seleccionado por su **pk**.

```python

@api_view(['GET','PUT','PATCH','DELETE'])
def details_products(request, pk):
    try:
        product= Product.objects.get(pk=pk)
    except Product.DoesNotExist:
        return Response(
            {'error': 'Product not found'},
            status=status.HTTP_404_NOT_FOUND
        )

    if request.method == 'GET':
        serializer = ProductSerializer(product)
        return Response(serializer.data)

    if request.method in['PUT, PATCH']:
        serializer = ProductSerializer(
            product,
            data = request.data,
            partial=(request.method == 'PATCH')
        )


        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data)
        return Response(
            serializer.errors,
            status=status.HTTP_400_BAD_REQUEST
        )

    if request.method == 'DELETE':
        product.delete()
        return Response(status=status.HTTP_204_NO_CONTENT)

```

Resultado: el endpoint de detalle ya permite consultar, actualizar parcialmente, reemplazar y eliminar un producto mediante GET, PATCH, PUT y DELETE.

Probar el endpoint en Postman

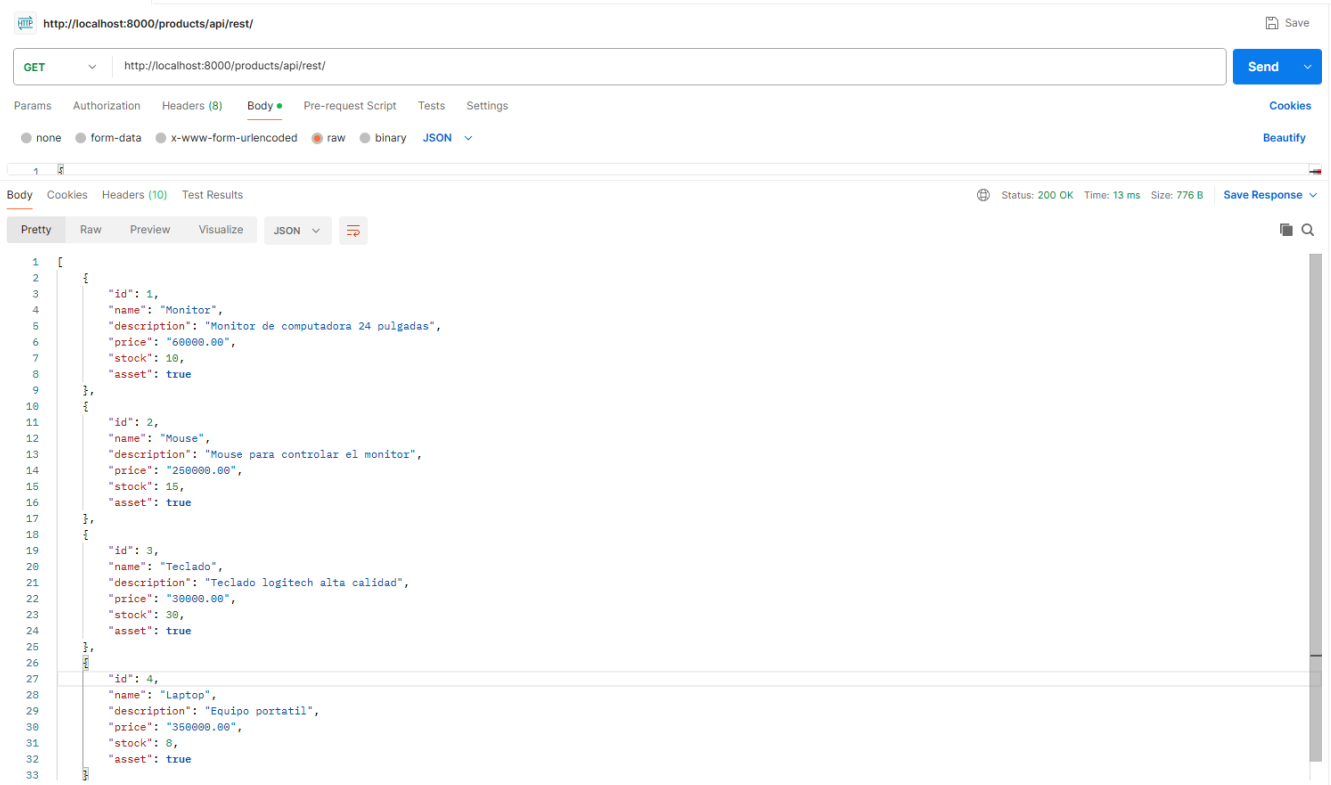
Paso 6 · Comprobar la respuesta de la API

Después de completar el endpoint, abrimos **Postman** para comprobar que la API responde correctamente. Seleccionamos el método, escribimos la URL y presionamos **Send**.

```
GET http://localhost:8000/products/api/rest/
```

En la captura se utiliza GET sobre la ruta de la colección. El estado **200 OK** confirma que la solicitud fue exitosa y el cuerpo de la respuesta muestra la lista de productos en formato JSON.

Para probar el endpoint de detalle creado en el Paso 6, agregamos el ID del producto al final de la dirección, por ejemplo **http://localhost:8000/products/api/rest/2/**. Sobre esa ruta podemos seleccionar GET, PUT, PATCH o DELETE según la operación que deseemos comprobar.



Agregar validaciones al serializer

Paso 7 · Validar precio y stock

Seguidamente, abrimos **products/serializers.py** y agregamos validaciones para impedir que se guarden precios o cantidades de stock incorrectas.

```
class ProductSerializer(serializers.ModelSerializer):
    class Meta:
        model = Product
        fields = '__all__'

    def validate_price(self, value):
        if value <= 0:
            raise serializers.ValidationError(
                'El precio debe ser mayor que cero.'
            )
        return value

    def validate_stock(self, value):
        if value < 0:
            raise serializers.ValidationError(
                'El stock no puede ser negativo'
            )
        return value
```

1. Nombre de los métodos: DRF reconoce automáticamente el patrón **validate_<campo>**. Por eso **validate_price** valida el campo price y **validate_stock** valida stock.

Cómo funcionan las validaciones

Paso 7 · Continuación en ProductSerializer

2. Parámetro value: contiene el dato que el cliente envió para ese campo. Antes de crear o actualizar el producto, el serializer pasa el valor por este método.

3. Validación del precio: si el precio es igual o menor que cero, **ValidationError** detiene la validación y devuelve el mensaje correspondiente al cliente.

4. Validación del stock: el stock puede ser cero, pero no puede ser negativo. Si es menor que cero, también se genera un error de validación.

5. return value: cuando el dato cumple la condición, se devuelve el mismo valor para que el serializer pueda continuar y guardarlo.

Los dos métodos deben quedar dentro de la clase **ProductSerializer**, al mismo nivel de indentación que la clase **Meta**.

Corrección mínima: en la captura, **validate_price** termina solamente con **return**. Debe escribirse **return value**; de lo contrario, el método devuelve **None** y el precio validado no se conserva correctamente.



Resultado: estas reglas se aplican automáticamente cuando llamamos serializer.is_valid() en solicitudes POST, PUT o PATCH. Si un valor es inválido, la API responde con los errores y el estado 400 Bad Request.